

1 0

10 Lists

10.0.1 Variables with more than one value

You have already seen ordinary variables that store a single value. However other variable types can hold more than one value. These are called containers because they can contain more than one object. The simplest type is called a list. Here is an example of a list being used:

```
which_one = int(input("What month (1-12)? "))
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
          'August', 'September', 'October', 'November', 'December']

if 1 <= which_one <= 12:
    print("The month is", months[which_one - 1])
```

and an output example:

```
What month (1-12)? 3
The month is March
```

In this example the `months` is a list. `months` is defined with the lines `months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']` (note that a `\` could also be used to split a long line, but that is not necessary in this case because Python is intelligent enough to recognize that everything within brackets belongs together). The `[` and `]` start and end the list with commas `,` separating the list items. The list is used in `months[which_one - 1]`. A list consists of items that are numbered starting at 0. In other words if you wanted January you would use `months[0]`. Give a list a number and it will return the value that is stored at that location.

The statement `if 1 <= which_one <= 12:` will only be true if `which_one` is between one and twelve inclusive (in other words it is what you would expect if you have seen that in algebra).

Lists can be thought of as a series of boxes. Each box has a different value. For example, the boxes created by `demolist = ['life', 42, 'the universe', 6, 'and', 9]` would look like this:

box number	0	1	2	3	4	5
demolist	"life"	42	"the universe"	6	"and"	9

Each box is referenced by its number so the statement `demolist[0]` would get `'life'`, `demolist[1]` would get 42 and so on up to `demolist[5]` getting 9.

10.0.2 More features of lists

The next example is just to show a lot of other stuff lists can do (for once I don't expect you to type it in, but you should probably play around with lists in interactive mode until you are comfortable with them.). Here goes:

```
demolist = ["life", 42, "the universe", 6, "and", 9]
print("demolist = ",demolist)
demolist.append("everything")
print("after 'everything' was appended demolist is now:")
print(demolist)
print("len(demolist) =", len(demolist))
print("demolist.index(42) =", demolist.index(42))
print("demolist[1] =", demolist[1])

# Next we will loop through the list
for c in range(len(demolist)):
    print("demolist[" + c + "] =", demolist[c])

del demolist[2]
print("After 'the universe' was removed demolist is now:")
print(demolist)
if "life" in demolist:
    print("'life' was found in demolist")
else:
    print("'life' was not found in demolist")

if "amoeba" in demolist:
    print("'amoeba' was found in demolist")

if "amoeba" not in demolist:
    print("'amoeba' was not found in demolist")

another_list = [42,7,0,123]
another_list.sort()
print("The sorted another_list is", another_list)
```

The output is:

```
demolist = ['life', 42, 'the universe', 6, 'and', 9]
after 'everything' was appended demolist is now:
['life', 42, 'the universe', 6, 'and', 9, 'everything']
len(demolist) = 7
demolist.index(42) = 1
demolist[1] = 42
demolist[ 0 ] = life
demolist[ 1 ] = 42
demolist[ 2 ] = the universe
demolist[ 3 ] = 6
demolist[ 4 ] = and
demolist[ 5 ] = 9
demolist[ 6 ] = everything
After 'the universe' was removed demolist is now:
['life', 42, 6, 'and', 9, 'everything']
'life' was found in demolist
'amoeba' was not found in demolist
The sorted another_list is [0, 7, 42, 123]
```

This example uses a whole bunch of new functions. Notice that you can just `print` a whole list. Next the `append` function is used to add a new item to the end of the list. `len` returns how many items are in a list. The valid indexes (as in numbers that can be used inside of the `[]`) of a list range from 0 to `len - 1`. The `index` function tells where the first location

of an item is located in a list. Notice how `demolist.index(42)` returns 1, and when `demolist[1]` is run it returns 42. To get help on all the functions a list provides for you, type `help(list)` in the interactive Python interpreter.

The line `# Next we will loop through the list` is just a reminder to the programmer (also called a *comment*). Python ignores everything that is written after a `#` on the current line. Next the lines:

```
for c in range(len(demolist)):
    print('demolist[' + c + '] =', demolist[c])
```

create a variable `c`, which starts at 0 and is incremented until it reaches the last index of the list. Meanwhile the `print` statement prints out each element of the list.

A much better way to do the above is:

```
for c, x in enumerate(demolist):
    print("demolist[" + c + "] =", x)
```

The `del` command can be used to remove a given element in a list. The next few lines use the `in` operator to test if an element is in or is not in a list. The `sort` function sorts the list. This is useful if you need a list in order from smallest number to largest or alphabetical. Note that this rearranges the list. In summary, for a list, the following operations occur:

example	explanation
<code>demolist[2]</code>	accesses the element at index 2
<code>demolist[2] = 3</code>	sets the element at index 2 to be 3
<code>del demolist[2]</code>	removes the element at index 2
<code>len(demolist)</code>	returns the length of <code>demolist</code>
<code>"value" in demolist</code>	is <i>True</i> if <code>"value"</code> is an element in <code>demolist</code>
<code>"value" not in demolist</code>	is <i>True</i> if <code>"value"</code> is not an element in <code>demolist</code>
<code>another_list.sort()</code>	sorts <code>another_list</code> . Note that the list must be all numbers or all strings to be sorted.
<code>demolist.index("value")</code>	returns the index of the first place that <code>"value"</code> occurs
<code>demolist.append("value")</code>	adds an element <code>"value"</code> at the end of the list
<code>demolist.remove("value")</code>	removes the first occurrence of <code>value</code> from <code>demolist</code> (same as <code>del demolist[demolist.index("value")]</code>)

This next example uses these features in a more useful way:

```
menu_item = 0
namelist = []
while menu_item != 9:
    print("-----")
    print("1. Print the list")
    print("2. Add a name to the list")
    print("3. Remove a name from the list")
    print("4. Change an item in the list")
    print("9. Quit")
    menu_item = int(input("Pick an item from the menu: "))
    if menu_item == 1:
        current = 0
        if len(namelist) > 0:
```

```

        while current < len(namelist):
            print(current, ".", namelist[current])
            current = current + 1
        else:
            print("List is empty")
    elif menu_item == 2:
        name = input("Type in a name to add: ")
        namelist.append(name)
    elif menu_item == 3:
        del_name = input("What name would you like to remove: ")
        if del_name in namelist:
            # namelist.remove(del_name) would work just as fine
            item_number = namelist.index(del_name)
            del namelist[item_number]
            # The code above only removes the first occurrence of
            # the name. The code below from Gerald removes all.
            # while del_name in namelist:
            #     item_number = namelist.index(del_name)
            #     del namelist[item_number]
        else:
            print(del_name, "was not found")
    elif menu_item == 4:
        old_name = input("What name would you like to change: ")
        if old_name in namelist:
            item_number = namelist.index(old_name)
            new_name = input("What is the new name: ")
            namelist[item_number] = new_name
        else:
            print(old_name, "was not found")

print("Goodbye")

```

And here is part of the output:

```

-----
1. Print the list
2. Add a name to the list
3. Remove a name from the list
4. Change an item in the list
9. Quit

Pick an item from the menu: 2
Type in a name to add: Jack

Pick an item from the menu: 2
Type in a name to add: Jill

Pick an item from the menu: 1
0 . Jack
1 . Jill

Pick an item from the menu: 3
What name would you like to remove: Jack

Pick an item from the menu: 4
What name would you like to change: Jill
What is the new name: Jill Peters

Pick an item from the menu: 1
0 . Jill Peters

Pick an item from the menu: 9
Goodbye

```

That was a long program. Let's take a look at the source code. The line `namelist = []` makes the variable `namelist` a list with no items (or elements). The next important line is `while menu_item != 9:`. This line starts a loop that allows the menu system for this program. The next few lines display a menu and decide which part of the program to run.

The section

```
current = 0
if len(namelist) > 0:
    while current < len(namelist):
        print(current, ".", namelist[current])
        current = current + 1
else:
    print("List is empty")
```

goes through the list and prints each name. `len(namelist)` tells how many items are in the list. If `len` returns 0, then the list is empty.

Then, a few lines later, the statement `namelist.append(name)` appears. It uses the `append` function to add an item to the end of the list. Jump down another two lines, and notice this section of code:

```
item_number = namelist.index(del_name)
del namelist[item_number]
```

Here the `index` function is used to find the index value that will be used later to remove the item. `del namelist[item_number]` is used to remove an element of the list.

The next section

```
old_name = input("What name would you like to change: ")
if old_name in namelist:
    item_number = namelist.index(old_name)
    new_name = input("What is the new name: ")
    namelist[item_number] = new_name
else:
    print(old_name, "was not found")
```

uses `index` to find the `item_number` and then puts `new_name` where the `old_name` was.

Congratulations, with lists under your belt, you now know enough of the language that you could do any computations that a computer can do (this is technically known as Turing-Completeness¹). Of course, there are still many features that are used to make your life easier.

10.0.3 Examples

test.py

```
## This program runs a test of knowledge

# First get the test questions
# Later this will be modified to use file io.
def get_questions():
    # notice how the data is stored as a list of lists
    return ["What color is the daytime sky on a clear day? ", "blue"],
```

1 https://en.wikipedia.org/wiki/Turing_completeness

```
        ["What is the answer to life, the universe and everything? ", "42"],
        ["What is a three letter word for mouse trap? ", "cat"]]

# This will test a single question
# it takes a single question in
# it returns True if the user typed the correct answer, otherwise False

def check_question(question_and_answer):
    # extract the question and the answer from the list
    # This function takes a list with two elements, a question and an answer.
    question = question_and_answer[0]
    answer = question_and_answer[1]
    # give the question to the user
    given_answer = input(question)
    # compare the user's answer to the tester's answer
    if answer == given_answer:
        print("Correct")
        return True
    else:
        print("Incorrect, correct was:", answer)
        return False

# This will run through all the questions
def run_test(questions):
    if len(questions) == 0:
        print("No questions were given.")
        # the return exits the function
        return
    index = 0
    right = 0
    while index < len(questions):
        # Check the question
        #Note that this is extracting a question and answer list from the list
        # of lists.
        if check_question(questions[index]):
            right = right + 1
        # go to the next question
        index = index + 1
    # notice the order of the computation, first multiply, then divide
    print("You got", right * 100 / len(questions),\
          "% right out of", len(questions))

# now let's get the questions from the get_questions function, and
# send the returned list of lists as an argument to the run_test function.

run_test(get_questions())
```

The values True and False point to 1 and 0, respectively. They are often used in sanity checks, loop conditions etc. You will learn more about this a little bit later (chapter Boolean Expressions²). Please note that `get_questions()` is essentially a list because even though it's technically a function, returning a list of lists is the only thing it does.

Sample Output:

```
What color is the daytime sky on a clear day? green
Incorrect, correct was: blue
What is the answer to life, the universe and everything? 42
```

² Chapter 12 on page 67

Correct
 What is a three letter word for mouse trap? **cat**
 Correct
 You got 66 % right out of 3

10.0.4 Exercises

Expand the test.py program so it has a menu giving the option of taking the test, viewing the list of questions and answers, and an option to quit. Also, add a new question to ask, "What noise does a truly advanced machine make?" with the answer of "ping".

Solution

Expand the test.py program so it has menu giving the option of taking the test, viewing the list of questions and answers, and an option to quit. Also, add a new question to ask, "What noise does a truly advanced machine make?" with the answer of "ping".

```
## This program runs a test of knowledge

questions = [["What color is the daytime sky on a clear day? ", "blue"],
             ["What is the answer to life, the universe and everything? ",
              "42"],
             ["What is a three letter word for mouse trap? ", "cat"],
             ["What noise does a truly advanced machine make?", "ping"]]

# This will test a single question
# it takes a single question in
# it returns True if the user typed the correct answer, otherwise False

def check_question(question_and_answer):
    # extract the question and the answer from the list
    question = question_and_answer[0]
    answer = question_and_answer[1]
    # give the question to the user
    given_answer = input(question)
    # compare the user's answer to the testers answer
    if answer == given_answer:
        print("Correct")
        return True
    else:
        print("Incorrect, correct was:", answer)
        return False

# This will run through all the questions

def run_test(questions):

    if len(questions) == 0:
        print("No questions were given.")
        # the return exits the function
        return
    index = 0
    right = 0
    while index < len(questions):
        # Check the question
        if check_question(questions[index]):
            right = right + 1
        # go to the next question
        index = index + 1
```

```
        index = index + 1
    # notice the order of the computation, first multiply, then divide
    print("You got", right * 100 / len(questions),
          "% right out of", len(questions))

#showing a list of questions and answers
def showquestions():
    q = 0
    while q < len(questions):
        a = 0
        print("Q:" , questions[q][a])
        a = 1
        print("A:" , questions[q][a])
        q = q + 1

# now let's define the menu function
def menu():
    print("-----")
    print("Menu:")
    print("1 - Take the test")
    print("2 - View a list of questions and answers")
    print("3 - View the menu")
    print("5 - Quit")
    print("-----")

choice = "3"
while choice != "5":
    if choice == "1":
        run_test(questions)
    elif choice == "2":
        showquestions()
    elif choice == "3":
        menu()
    print()
    choice = input("Choose your option from the menu above: ")
```

11 For Loops

With a for loop, we can repeat a piece of code. Let's look at an example:

```
one_to_ten = range(1, 11)
for count in one_to_ten:
    print(count)
```

The output of this code looks as follows:

```
1
2
3
4
5
6
7
8
9
10
```

The output looks awfully familiar but the program code looks different. The first line uses the `range` function. The `range` function uses two arguments like this `range(start, finish)`. `start` is the first number that is produced. `finish` is one larger than the last number. Note that this program could have been done in a shorter way:

```
for count in range(1, 11):
    print(count)
```

The `range` function returns an iterable. This can be converted into a list with the `list` function. which will then be the dominant number. Here are some examples to show what happens with the `range` command:

```
>>> range(1, 10)
range(1, 10)
>> list(range(1, 10))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>> list(range(-32, -20))
[-32, -31, -30, -29, -28, -27, -26, -25, -24, -23, -22, -21]
>> list(range(5,21))
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
>> list(range(5))
[0, 1, 2, 3, 4]
>> list(range(21, 5))
[]
```

The next line `for count in one_to_ten:` uses the `for` control structure. A `for` control structure looks like `for variable in list:`. `list` is gone through starting with the first element of the list and going to the last. As `for` goes through each element in a list it puts

each into **variable**. That allows **variable** to be used in each successive time the **for** loop is run through. Here is another example (you don't have to type this) to demonstrate:

```
demolist = ['life', 42, 'the universe', 6, 'and', 7, 'everything']
for item in demolist:
    print("The current item is:",item)
```

The output is:

```
The current item is: life
The current item is: 42
The current item is: the universe
The current item is: 6
The current item is: and
The current item is: 7
The current item is: everything
```

Notice how the **for** loop goes through and sets **item** to each element in the list. So, what is **for** good for? The first use is to go through all the elements of a list and do something with each of them. Here's a quick way to add up all the elements:

```
list = [2, 4, 6, 8]
sum = 0
for num in list:
    sum = sum + num

print("The sum is:", sum)
```

with the output simply being:

```
The sum is: 20
```

Or you could write a program to find out if there are any duplicates in a list like this program does:

```
list = [4, 5, 7, 8, 9, 1, 0, 7, 10]
list.sort()
prev = None
for item in list:
    if prev == item:
        print("Duplicate of", prev, "found")
    prev = item
```

and for good measure:

```
Duplicate of 7 found
```

Okay, so how does it work? Here is a special debugging version to help you understand (you don't need to type this in):

```
l = [4, 5, 7, 8, 9, 1, 0, 7, 10]
print("l = [4, 5, 7, 8, 9, 1, 0, 7, 10]", "\t\tl:", l)
l.sort()
print("l.sort()", "\t\tl:", l)
```

```

prev = l[0]
print("prev = l[0]", "\t\tprev:", prev)
del l[0]
print("del l[0]", "\t\tl:", l)
for item in l:
    if prev == item:
        print("Duplicate of", prev, "found")
    print("if prev == item:", "\t\tprev:", prev, "\t\titem:", item)
    prev = item
    print("prev = item", "\t\tprev:", prev, "\t\titem:", item)

```

with the output being:

l = [4, 5, 7, 8, 9, 1, 0, 7, 10]	l: [4, 5, 7, 8, 9, 1, 0, 7, 10]
l.sort()	l: [0, 1, 4, 5, 7, 7, 8, 9, 10]
prev = l[0]	prev: 0
del l[0]	l: [1, 4, 5, 7, 7, 8, 9, 10]
if prev == item:	prev: 0 item: 1
prev = item	prev: 1 item: 1
if prev == item:	prev: 1 item: 4
prev = item	prev: 4 item: 4
if prev == item:	prev: 4 item: 5
prev = item	prev: 5 item: 5
if prev == item:	prev: 5 item: 7
prev = item	prev: 7 item: 7
Duplicate of 7 found	
if prev == item:	prev: 7 item: 7
prev = item	prev: 7 item: 7
if prev == item:	prev: 7 item: 8
prev = item	prev: 8 item: 8
if prev == item:	prev: 8 item: 9
prev = item	prev: 9 item: 9
if prev == item:	prev: 9 item: 10
prev = item	prev: 10 item: 10

The reason I put so many `print` statements in the code was so that you can see what is happening in each line. (By the way, if you can't figure out why a program is not working, try putting in lots of print statements in places where you want to know what is happening.) First the program starts with a boring old list. Next the program sorts the list. This is so that any duplicates get put next to each other. The program then initializes a `prev`(ious) variable. Next the first element of the list is deleted so that the first item is not incorrectly thought to be a duplicate. Next a `for` loop is gone into. Each item of the list is checked to see if it is the same as the previous. If it is a duplicate was found. The value of `prev` is then changed so that the next time the `for` loop is run through `prev` is the previous item to the current. Sure enough, the 7 is found to be a duplicate. (Notice how `\t` is used to print a tab.)

The other way to use `for` loops is to do something a certain number of times. Here is some code to print out the first 9 numbers of the Fibonacci series:

```

a = 1
b = 1
for c in range(1, 10):
    print(a, end=" ")
    n = a + b
    a = b
    b = n

```

with the surprising output:

```
1 1 2 3 5 8 13 21 34
```

Everything that can be done with `for` loops can also be done with `while` loops but `for` loops give an easy way to go through all the elements in a list or to do something a certain number of times.

12 Boolean Expressions

Here is a little example of boolean expressions (you don't have to type it in):

```
a = 6
b = 7
c = 42
print(1, a == 6)
print(2, a == 7)
print(3, a == 6 and b == 7)
print(4, a == 7 and b == 7)
print(5, not a == 7 and b == 7)
print(6, a == 7 or b == 7)
print(7, a == 7 or b == 6)
print(8, not (a == 7 and b == 6))
print(9, not a == 7 and b == 6)
```

With the output being:

```
1 True
2 False
3 True
4 False
5 True
6 True
7 False
8 True
9 False
```

What is going on? The program consists of a bunch of funny looking **print** statements. Each **print** statement prints a number and an expression. The number is to help keep track of which statement I am dealing with. Notice how each expression ends up being either **False** or **True**. In Python false can also be written as 0 and true as 1.

The lines:

```
print(1, a == 6)
print(2, a == 7)
```

print out a **True** and a **False** respectively just as expected since the first is true and the second is false. The third print, **print(3, a == 6 and b == 7)**, is a little different. The operator **and** means if both the statement before and the statement after are true then the whole expression is true otherwise the whole expression is false. The next line, **print(4, a == 7 and b == 7)**, shows how if part of an **and** expression is false, the whole thing is false. The behavior of **and** can be summarized as follows:

expression	result
true and true	true

expression	result
true and false	false
false and true	false
false and false	false

Notice that if the first expression is false Python does not check the second expression since it knows the whole expression is false. Try running `False and print("Hi")` and compare this to running `True and print("Hi")`. The technical term for this is short-circuit evaluation¹

The next line, `print(5, not a == 7 and b == 7)`, uses the `not` operator. `not` just gives the opposite of the expression. (The expression could be rewritten as `print(5, a != 7 and b == 7)`). Here is the table:

expression	result
not true	false
not false	true

The two following lines, `print(6, a == 7 or b == 7)` and `print(7, a == 7 or b == 6)`, use the `or` operator. The `or` operator returns true if the first expression is true, or if the second expression is true or both are true. If neither are true it returns false. Here's the table:

expression	result
true or true	true
true or false	true
false or true	true
false or false	false

Notice that if the first expression is true Python doesn't check the second expression since it knows the whole expression is true. This works since `or` is true if at least one half of the expression is true. The first part is true so the second part could be either false or true, but the whole expression is still true.

The next two lines, `print(8, not (a == 7 and b == 6))` and `print(9, not a == 7 and b == 6)`, show that parentheses can be used to group expressions and force one part to be evaluated first. Notice that the parentheses changed the expression from false to true. This occurred since the parentheses forced the `not` to apply to the whole expression instead of just the `a == 7` portion.

Here is an example of using a boolean expression:

```
list = ["Life", "The Universe", "Everything", "Jack", "Jill", "Life", "Jill"]

# make a copy of the list. See the More on Lists chapter to explain what [:]
# means.
copy = list[:]
# sort the copy
copy.sort()
```

¹ <https://en.wikipedia.org/wiki/Short-circuit%20evaluation>


```
prev = copy[0]
del copy[0]

count = 0

# go through the list searching for a match
while count < len(copy) and copy[count] != prev:
    prev = copy[count]
    count = count + 1

# If a match was not found then count can't be < len
# since the while loop continues while count is < len
# and no match is found

if count < len(copy):
    print("First Match:", prev)
```

And here is the output:

```
First Match: Jill
```

This program works by continuing to check for match `while count < len(copy) and copy[count] is not equal to prev`. When either `count` is greater than the last index of `copy` or a match has been found the `and` is no longer true so the loop exits. The `if` simply checks to make sure that the `while` exited because a match was found.

The other "trick" of `and` is used in this example. If you look at the table for `and` notice that the third entry is "false and false". If `count >= len(copy)` (in other words `count < len(copy)` is false) then `copy[count]` is never looked at. This is because Python knows that if the first is false then they can't both be true. This is known as a short circuit and is useful if the second half of the `and` will cause an error if something is wrong. I used the first expression (`count < len(copy)`) to check and see if `count` was a valid index for `copy`. (If you don't believe me remove the matches "Jill" and "Life", check that it still works and then reverse the order of `count < len(copy) and copy[count] != prev` to `copy[count] != prev and count < len(copy)`.)

Boolean expressions can be used when you need to check two or more different things at once.

12.0.1 A note on Boolean Operators

A common mistake for people new to programming is a misunderstanding of the way that boolean operators works, which stems from the way the python interpreter reads these expressions. For example, after initially learning about "and" and "or" statements, one might assume that the expression `x == ('a' or 'b')` would check to see if the variable `x` was equivalent to one of the strings 'a' or 'b'. This is not so. To see what I'm talking about, start an interactive session with the interpreter and enter the following expressions:

```
>>> 'a' == ('a' or 'b')
>>> 'b' == ('a' or 'b')
```

```
>>> 'a' == ('a' and 'b')
>>> 'b' == ('a' and 'b')
```

And this will be the unintuitive result:

```
>>> 'a' == ('a' or 'b')
True
>>> 'b' == ('a' or 'b')
False
>>> 'a' == ('a' and 'b')
False
>>> 'b' == ('a' and 'b')
True
```

At this point, the `and` and `or` operators seem to be broken. It doesn't make sense that, for the first two expressions, `'a'` is equivalent to `'a' or 'b'` while `'b'` is not. Furthermore, it doesn't make any sense that `'b'` is equivalent to `'a' and 'b'`. After examining what the interpreter does with boolean operators, these results do in fact exactly what you are asking of them, it's just not the same as what you think you are asking.

When the Python interpreter looks at an `or` expression, it takes the first statement and checks to see if it is true. If the first statement is true, then Python returns that object's value without checking the second statement. This is because for an `or` expression, the whole thing is true if one of the values is true; the program does not need to bother with the second statement. On the other hand, if the first value is evaluated as false Python checks the second half and returns that value. That second half determines the truth value of the whole expression since the first half was false. This "laziness" on the part of the interpreter is called "short circuiting" and is a common way of evaluating boolean expressions in many programming languages.

Similarly, for an `and` expression, Python uses a short circuit technique to speed truth value evaluation. If the first statement is false then the whole thing must be false, so it returns that value. Otherwise if the first value is true it checks the second and returns that value.

One thing to note at this point is that the boolean expression returns a value indicating `True` or `False`, but that Python considers a number of different things to have a truth value assigned to them. To check the truth value of any given object `x`, you can use the function `bool(x)` to see its truth value. Below is a table with examples of the truth values of various objects:

True	False
True	False
1	0
Numbers other than zero	The string 'None'
Nonempty strings	Empty strings
Nonempty lists	Empty lists
Nonempty dictionaries	Empty dictionaries

Now it is possible to understand the perplexing results we were getting when we tested those boolean expressions before. Let's take a look at what the interpreter "sees" as it goes through that code:

First case:

```

»> 'a' == ('a' or 'b') # Look at parentheses first, so evaluate expression "('a' or 'b')"
```

'a' is a nonempty string, so the first value is

True

Return that first value: 'a'

```

»> 'a' == 'a' # the string 'a' is equivalent to the string 'a', so expression
is True
True
```

Second case:

```

»> 'b' == ('a' or 'b') # Look at parentheses first, so evaluate expression "('a' or 'b')"
```

'a' is a nonempty string, so the first value is

True

Return that first value: 'a'

```

»> 'b' == 'a' # the string 'b' is not equivalent to the string 'a', so
expression is False
False
```

Third case:

```

»> 'a' == ('a' and 'b') # Look at parentheses first, so evaluate expression "('a' and 'b')"
```

'a' is a nonempty string, so the first value is

True, examine second value

'b' is a nonempty string, so second value is True

Return that second value as result of whole

expression: 'b'

```

»> 'a' == 'b' # the string 'a' is not equivalent to the string 'b', so
expression is False
False
```

Fourth case:

```

»> 'b' == ('a' and 'b') # Look at parentheses first, so evaluate expression "('a' and 'b')"
```

'a' is a nonempty string, so the first value is

True, examine second value

'b' is a nonempty string, so second value is True

Return that second value as result of whole

expression: 'b'

```

»> 'b' == 'b' # the string 'b' is equivalent to the string 'b', so expression
is True
True
```

So Python was really doing its job when it gave those apparently bogus results. As mentioned previously, the important thing is to recognize what value your boolean expression will return when it is evaluated, because it isn't always obvious.

Going back to those initial expressions, this is how you would write them out so they behaved in a way that you want:

```
>>> 'a' == 'a' or 'a' == 'b'
True
>>> 'b' == 'a' or 'b' == 'b'
True
>>> 'a' == 'a' and 'a' == 'b'
False
>>> 'b' == 'a' and 'b' == 'b'
False
```

When these comparisons are evaluated they return truth values in terms of True or False, not strings, so we get the proper results.

12.0.2 Examples

password1.py

```
## This program asks a user for a name and a password.
# It then checks them to make sure that the user is allowed in.

name = input("What is your name? ")
password = input("What is the password? ")
if name == "Josh" and password == "Friday":
    print("Welcome Josh")
elif name == "Fred" and password == "Rock":
    print("Welcome Fred")
else:
    print("I don't know you.")
```

Sample runs

```
What is your name? Josh
What is the password? Friday
Welcome Josh
```

```
What is your name? Bill
What is the password? Money
I don't know you.
```

12.0.3 Exercises

Write a program that has a user guess your name, but they only get 3 chances to do so until the program quits.

Solution

```
print("Try to guess my name!")
count = 1
name = "guilherme"
guess = input("What is my name? ")
```